

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ «ДНІПРОВСЬКА
ПОЛІТЕХНІКА»



ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра системного аналізу та управління

ПРАКТИЧНА РОБОТА № 3

з дисципліни «Аналіз програмного забезпечення»

на тему: «Написання тест-кейсів (Test Case)»

Виконав:

здобувач групи 121-23-1

Земцов Олександр

Прийняв:

Шевченко Ю. О.

Дніпро

2026

Тема: Написання тест-кейсів (Test Case).

Мета: закріпити теоретичні знання і розвинути практичні навички із роботи QA\QC.

Завдання: придумайте об'єкт тестування. Це повинен бути будь-який об'єкт, який складається мінімум з 5 частин. (годинник, скейт, велосипед, монітор і т.п.) Напишіть не менше 10 тест кейсів до цього об'єкту, що зможуть оцінити його якість з точки зору продукту. В звіті роботи коротко опишіть ваш об'єкт і тест-кейси.

Обраний для тестування об'єкт:

Програма обробки даних із файлу у вигляді однозв'язного списку. Вхідний файл містить послідовність цілих чисел, які необхідно зчитати та сформувати з них список, що задовольняє наступним умовам:

- Усі парні числа повинні розміщуватися на початку списку;
- Усі непарні числа повинні розміщуватися після всіх парних чисел;
- Відносний порядок парних чисел між собою має залишатися таким самим, як у вхідному файлі;
- Відносний порядок непарних чисел між собою також має залишатися таким самим, як у файлі.

Тест-кейс: TC-001. **Перевірка створення списку з парними та непарними числами.**

Пріоритет: High (Високий). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «7 2 1 2 4 6 1 5 9 6».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test1.dat. Де test1.dat – підготовлений файл з тестовими даними
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «2 2 4 6 6 7 1 1 5 9».
Постумови (Post-conditions)	Програма коректно завершила роботу

Тест-кейс: TC-002. **Перевірка роботи функції з порожнім файлом.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Створений порожній текстовий файл.
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test1.dat. Де test1.dat – підготовлений файл з тестовими даними
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «».
Постумови (Post-conditions)	Програма коректно завершила роботу

Тест-кейс: TC-003. **Перевірка роботи функції з одним елементом.**

Пріоритет: Low (Низький). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «8».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test1.dat. Де test1.dat – підготовлений файл з тестовими даними
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «8».
Постумови (Post-conditions)	Програма коректно завершила роботу

Тест-кейс: TC-004. **Перевірка списку лише з парними числами.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «6 4 8 10 10 8 4 6 2 2».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test1.dat. Де test1.dat – підготовлений файл з тестовими даними
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «6 4 8 10 10 8 4 6 2 2».
Постумови (Post-conditions)	Програма коректно завершила роботу

Тест-кейс: TC-005. **Перевірка списку лише з непарними числами.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний).

Компонент: Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «7 3 15 1 9 11».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test5.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «7 3 15 1 9 11 ». Порядок елементів залишається незмінним.
Постумови (Post-conditions)	Програма коректно завершила роботу

Тест-кейс: TC-006. **Перевірка списку лише з парними числами.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний).

Компонент: Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «-3 -4 0 5 -2 1».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test6.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «-4 0 -2 -3 5 1 ». Парні числа (-4, 0, -2) розміщені першими, непарні (-3, 5, 1) – після них.
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-007. **Перевірка чергування парних та непарних чисел.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний).

Компонент: Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «1 2 3 4 5 6».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test7.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «2 4 6 1 3 5 ».
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-008. **Перевірка поведінки при наявності нечислових даних.**

Пріоритет: High (Високий). **Тип:** Negative (Негативний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із некоректними даними: «2 4 text 5 7».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test8.dat
Очікуваний результат (Expected result)	Зчитування зупиняється на нечисловому символі text (fscanf повертає 0). Програма повертає зчитану до помилки частину: «2 4 » та без аварійного завершення (crash/segfault) звільняє пам'ять.
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-009. **Перевірка обробки нулів у вхідних даних.**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний).
Компонент: Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із таким вмістом: «0 3 0 5 0».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test9.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «0 0 3 5 ». Нулі трактуються як парні числа і розміщуються на початку списку.
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-010. **Перевірка граничних значень цілих чисел (INT_MAX, INT_MIN).**

Пріоритет: High (Високий). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із максимальним та мінімальним 32-бітними значеннями: «2147483647 -2147483648 0 1».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test10.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «-2147483648 0 2147483647 1 ». Значення INT_MIN та 0 оброблені як парні, INT_MAX та 1 — як непарні, без переповнення.
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-011. **Перевірка обробки нетипових розділювачів.**

Пріоритет: Low (Низький). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із числами, розділеними знаками табуляції \t та декількома переносами рядків \n: «12\t7\n\n4\n3».
Кроки (Steps)	1. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test11.dat
Очікуваний результат (Expected result)	Програма коректно ігнорує будь-які пробільні символи та повертає в консоль: «12 4 7 3 ».
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Тест-кейс: TC-012. **Перевірка збереження порядкового номера однакових значень (дублікатів).**

Пріоритет: Medium (Середній). **Тип:** Positive (Позитивний). **Компонент:** Data Processing Module. **Пристрій:** Будь-який, що підтримує компілятор мови C.

Поле	Опис
Передумови (Pre-conditions)	1. Додаток встановлено на пристрій 2. Підготовлений текстовий файл із серією однакових парних та непарних чисел: «5 5 2 2 5 2».
Кроки (Steps)	2. Запустити програму, подаючи на стандартний ввід підготовлений файл. Приклад команди: solution_al.exe < my_tests\test12.dat
Очікуваний результат (Expected result)	Програма повертає на стандартний вивід (в консоль): «2 2 2 5 5 5 ». Усі дублікати зберегли свої відносні позиції в межах своїх груп.
Постумови (Post-conditions)	Програма коректно завершила роботу, пам'ять звільнено

Висновок:

Під час виконання практичної роботи було закріплено теоретичні знання і розвинуто практичні навички із роботи QA\QC. Набуто вміння створювати, тест-кейси для оцінки об'єкту з точки зору продукту.